

AERIALITH

Secure Kubernetes Production Architecture

Self-hosted HA platform on Proxmox
Talos Linux + Kubernetes + Cilium + Istio Ambient

Implementation Architecture v3.0
30 August 2026 | Idaho, United States | Confidential



Decision summary: replace the K3s-centric baseline with a hardened Talos Linux / upstream Kubernetes design; separate control-plane, platform, and application node pools; use Cilium for CNI and L3/L4 security, Istio Ambient for workload identity and L7 policy, Flux + Flagger for GitOps canaries, and a defense-in-depth security toolchain aligned to ISO 27001, SOC 2, HIPAA, and GDPR evidence needs.

Document status

Supersedes v2.0

This document supersedes the uploaded Aerealith Self-Hosted HA Production Architecture v2.0. The biggest technical change is the Kubernetes distribution: K3s is no longer the default. The selected baseline is Talos Linux running upstream Kubernetes because the new priority is security depth rather than minimal resource footprint.

This is an implementation architecture and security baseline. It supports audit readiness for ISO/IEC 27001, SOC 2, HIPAA Security Rule safeguards, and GDPR accountability, but it does not create certification, attestation, or legal compliance by itself.

Field	Value
Architecture owner	TBD
Security approver	TBD
Service owner	Aerealith product owner - TBD
Version	v3.0
Review cadence	Quarterly and after material architecture, threat, data-classification, or regulatory change
Change control	Protected Git pull request with retained validation and approval evidence

Table of contents

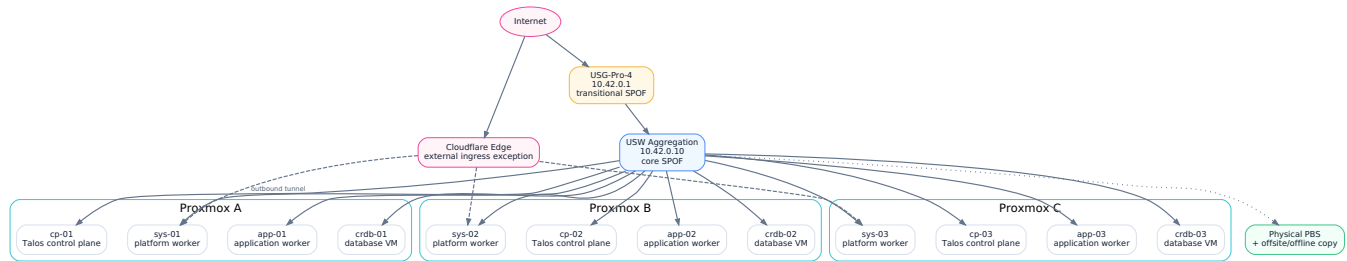
1. Executive decision	4
2. Kubernetes distribution selection	5
3. Threat model and trust boundaries	6
4. Physical placement and node pools	8
5. Address plan and network services	9
6. Talos Linux host hardening	10
7. Control-plane hardening	11
8. API, etcd, and node firewall policy	13
9. Cilium network security plane	14
10. Istio Ambient service security	15
11. Namespaces, node pools, and security zones	16
12. Default-deny network policy model	17
13. Identity, RBAC, and administrative access	18

14. Pod Security and Kyverno admission policy	19
15. Software supply-chain security	20
16. Self-hosted OCI registry strategy	21
17. Secrets management and PKI	22
18. Canary and progressive delivery	23
19. Flux GitOps operating model	24
20. Cloudflare Tunnel and private ingress	25
21. Runtime security and posture management	26
22. Observability and telemetry architecture	27
23. Audit logging and evidence-quality telemetry	28
24. CockroachDB security and placement	29
25. Storage and persistence strategy	30
26. Backup and disaster recovery	31
27. Failure-mode analysis	32
28. Incident containment and recovery	33
29. Patch, upgrade, and vulnerability management	34
30. Vulnerability-management SLA	35
31. Security standards and compliance mapping	36
32. Automated compliance evidence	37
33. Phased implementation plan	38
34. Production validation checklist	39
35. Definitive architecture	40
A. Configuration baselines	41
B. Port and firewall matrix	42
C. Component maturity and selection	43
D. Authoritative references	44

1. Executive decision

Aerealith should run on a deliberately hardened Kubernetes platform where the node operating system is treated as part of the security boundary, not as an unmanaged substrate. The recommended baseline is Talos Linux + upstream Kubernetes, not K3s. Talos removes SSH, interactive shells, package management, and mutable root filesystem state; management is API-driven and authenticated with mTLS. ^{[R1][R2]}

The cluster uses three dedicated control-plane VMs, three platform workers, and three application workers, spread one of each across the three physical Proxmox hosts. If hardware cannot sustain that topology with N+1 headroom, the temporary minimum is three combined workers, but that is a reduced-isolation mode and should be recorded as a risk exception.



Recommended physical failure-domain layout. Quorum members and worker pools are spread one per Proxmox host.

Layer	Selected design	Security purpose
Node OS	Talos Linux	Immutable/minimal host; no SSH or shell; API-only administration
Kubernetes	Upstream Kubernetes on Talos	Standard APIs and conformance without a lightweight-distribution constraint
Control plane	3 dedicated VMs + embedded etcd	2/3 quorum after one physical-host loss
Platform workers	3 VMs	Isolate cluster/security services from product load
Application workers	3 VMs	Aerealith workloads; resource and blast-radius separation
CNI	Cilium	L3/L4 identity policy, Hubble, encryption
Service mesh	Istio Ambient	Workload identity, mTLS, L7 authorization and traffic shifting
GitOps	FluxCD	Pull-based production reconciliation
Progressive delivery	Flagger	Prometheus-gated canary promotion/rollback
Admission	Pod Security Admission + Kyverno	Prevent unsafe workload configurations and unsigned images
Runtime security	Falco + Kubescape; optional Tetragon	Behavior detection + posture/compliance evidence
Secrets/PKI	SOPS bootstrap + OpenBao + cert-manager	Separate bootstrap and runtime secret authorities
Public ingress	Cloudflare Tunnel -> private Istio Gateway	No public origin listener or NodePort
Database	CockroachDB on 3 dedicated VMs	Database lifecycle remains independent from Kubernetes
Backups	Physical PBS + offsite/offline copy	Independent recovery plane

2. Kubernetes distribution selection

The v2.0 document made K3s the default. That was reasonable for a small footprint, but the current requirement is an in-depth secure Kubernetes cluster. The distribution decision should therefore optimize host attack surface, configuration drift, CIS posture, controlled upgrades, and recovery rather than installation convenience.

Option	Security posture	Operational model	Strengths	Main drawback	Decision
Talos Linux + Kubernetes	Excellent by design	API-driven immutable nodes	No SSH/shell/package manager; default hardening; built-in VIP; atomic upgrades	Different operating model; fewer traditional host agents	SELECTED
RKE2	Very strong	Traditional Linux + hardened distro	CIS profile; familiar systemd/SSH tooling; hardened defaults	Host OS remains a large mutable security surface	Fallback
kubeadm	Depends on operator	Upstream primitives	Maximum upstream transparency and flexibility	Highest hardening/upgrade burden	Not selected
K3s	Good when hardened	Lightweight integrated distro	Small footprint and easy HA	Resource savings are no longer the primary design goal	Not selected for production baseline

Why Talos wins

Talos explicitly documents no SSH, no interactive shell, an immutable root filesystem, no package manager, API-only management, Kubernetes audit logging, secret encryption at rest, mTLS to etcd, disabled anonymous API authentication, and strong defaults.^{[R1][R2]}

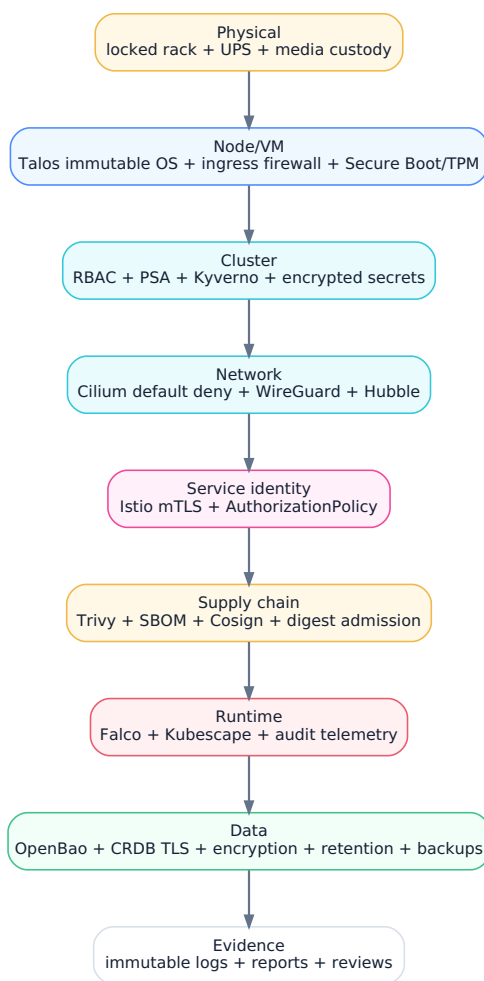
RKE2 remains the recovery option if a future requirement mandates conventional Linux host agents, custom kernel tooling, or an enterprise support workflow that is easier on Ubuntu/RHEL. Its current hardening guide is explicitly designed around CIS benchmark conformance.^[R22]

Kubeadm remains useful as the reference implementation for understanding Kubernetes HA, but it deliberately leaves host lifecycle and most operational choices to the operator.^[R21]

3. Threat model and trust boundaries

The secure design starts from the failure and attacker model. A flat LAN means a compromised workstation, appliance, printer, BMC, or VM can reach the L2 address of every other device unless the destination rejects it. Kubernetes NetworkPolicy does not protect Proxmox, CockroachDB VMs, Talos APIs, network appliances, or BMCs.

Threat / failure	Primary controls	Residual risk
Compromised LAN client	Talos ingress firewall; Proxmox VM firewall; WireGuard-only admin; service TLS	Same L2 remains discoverable; gateway cannot filter same-subnet traffic
Compromised workload	Cilium default deny; Istio mTLS/authz; non-root; seccomp; Kyverno; runtime detection	Kernel/container-runtime zero-days
Malicious or compromised developer token	Protected branches; CODEOWNERS; CI least privilege; signed artifacts; production GitOps approval	Git is still a Tier-0 trust plane
Compromised image/dependency	Trivy; SBOM; Cosign; digest pinning; Kyverno verification	Unknown supply-chain attack not detected by scanners
One Proxmox host lost	etcd/CRDB/OpenBao quorum + topology spread + N+1 workers	Second correlated host loss
Control-plane credential theft	mTLS; Talos API restricted to management overlay; short admin kubeconfig; offline break-glass	Tier-0 credential exposure can be severe
Ransomware/operator deletion	Immutable/offline backups; delete-separated credentials; GitOps rebuild	Site-wide compromise before backup isolation
Cloudflare outage/account issue	Private management plane remains; provider monitoring; documented exception	Public app ingress is unavailable without alternate edge



Defense-in-depth layers. No single product carries the security claim.

4. Physical placement and node pools

The preferred production topology uses nine Kubernetes VMs: three control-plane, three platform workers, and three application workers. This is intentionally more expensive in RAM than v2.0. The benefit is stronger workload isolation, clearer resource ownership, safer maintenance, and reduced probability that product load starves security, ingress, secret, or observability services.

Node	Role	PVE placement	Planning vCPU	Planning RAM	Planning disk	Scheduling
k8s-cp-01	Talos control plane + etcd	PVE A	4	8	120 GiB SSD/NVMe	No normal workloads
k8s-cp-02	Talos control plane + etcd	PVE B	4	8	120 GiB SSD/NVMe	No normal workloads
k8s-cp-03	Talos control plane + etcd	PVE C	4	8	120 GiB SSD/NVMe	No normal workloads
k8s-sys-01	Platform worker	PVE A	8	24	100 GiB OS + 300-500 GiB data	Platform/security only
k8s-sys-02	Platform worker	PVE B	8	24	100 GiB OS + 300-500 GiB data	Platform/security only
k8s-sys-03	Platform worker	PVE C	8	24	100 GiB OS + 300-500 GiB data	Platform/security only
k8s-app-01	Application worker	PVE A	12	48	100 GiB OS + 500 GiB-1 TiB	Aerealith workloads
k8s-app-02	Application worker	PVE B	12	48	100 GiB OS + 500 GiB-1 TiB	Aerealith workloads
k8s-app-03	Application worker	PVE C	12	48	100 GiB OS + 500 GiB-1 TiB	Aerealith workloads

Capacity gate

Do not shrink the security topology merely to fit current hardware. First inventory actual CPU, RAM, disk endurance, and workload requests. Production is accepted only if two surviving application workers and two surviving platform workers can carry critical requests after a complete Proxmox-host loss.

If the physical hosts cannot support the nine-VM plan, use three combined workers temporarily and create a documented capacity/security remediation item. Do not move normal workloads onto the control plane.

5. Address plan and network services

The physical LAN stays 10.42.0.0/24 and remains intentionally unsegmented by VLAN. Kubernetes must therefore avoid the K3s-default ranges already used on the LAN. The current selected Pod and Service ranges remain valid unless another site/VPN route conflicts.

Use	Range / value	Status
Physical LAN	10.42.0.0/24	Existing
Pod CIDR	10.200.0.0/16	Selected
Service CIDR	10.201.0.0/16	Selected
Cluster DNS	10.201.0.10	Selected
Kubernetes API VIP	10.42.0.TBD	Reserve outside DHCP
Talos API	TCP 50000 on each node	Management overlay only
Management overlay	10.250.0.0/24 proposed	Validate all remote routes
CockroachDB	10.42.0.TBD x3	Dedicated VMs; static

Talos provides a built-in layer-2 shared VIP for the Kubernetes API. It uses etcd election to decide which control-plane node owns the IP and requires the control-plane nodes to share the same L2 network. ^[R3]

Important Talos recovery rule

Do not use the Kubernetes VIP as the Talos API endpoint for recovery. Talos documents that the VIP depends on etcd; direct per-node Talos API addresses must remain reachable from the management overlay even when etcd is unhealthy. ^[R3]

6. Talos Linux host hardening

Talos is the largest security improvement over v2.0. Traditional hardening lists spend substantial effort disabling SSH, limiting shell access, maintaining packages, reducing installed software, protecting systemd services, and detecting configuration drift. Talos removes most of those surfaces by design. ^{[R1][R2]}

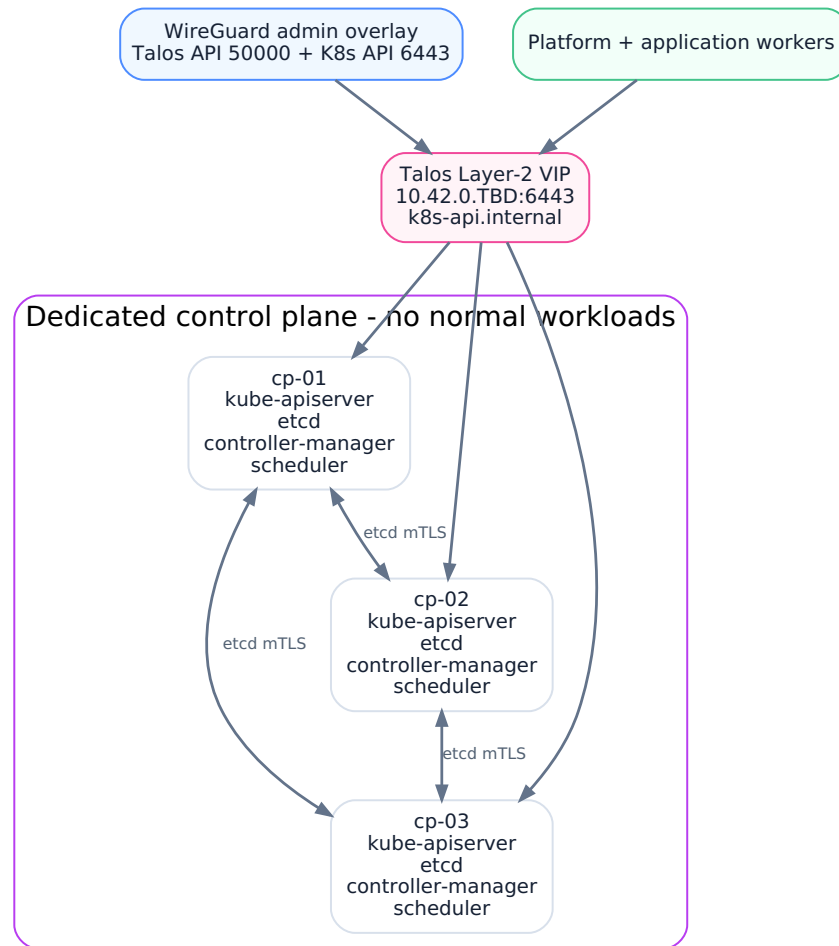
Host control	Production requirement
Remote administration	Talos API only; mTLS client identities; reachable only from management overlay and named recovery station
Root filesystem	Immutable; no package manager; no ad-hoc node changes
Ingress firewall	Default block using Talos NetworkDefaultActionConfig; explicit port/source rules only
Disk encryption	Encrypt STATE and EPHEMERAL; prefer TPM-backed LUKS2 where validated
Secure Boot	Enable UEFI Secure Boot and TPM/vTPM after Proxmox proof-of-concept; preserve signing/recovery keys
Logging	Forward Talos service and kernel logs to independent collector
Node config	Machine configuration generated from reviewed patches; secrets kept out of normal Git
Upgrades	One node at a time; preflight; A/B rollback; never skip supported minor paths
Debug access	Time-bounded privileged debug container only under incident/change record; not a routine shell substitute

Talos supports Secure Boot with UKI and TPM-bound disk encryption, and its disk-encryption guidance distinguishes stronger TPM/KMS methods from weaker static or node-derived options. ^{[R33][R34]}

Production Proxmox VM firmware

Use OVMF/UEFI, q35 machine type, CPU type host, fixed RAM for control-plane/DB VMs, VirtIO storage/networking, and a vTPM2 device where supported. Validate Secure Boot + vTPM + PBS restore before enforcing it everywhere.

7. Control-plane hardening



Control-plane availability uses Talos built-in L2 VIP and three etcd members, one per physical host.

Only Kubernetes control-plane components, etcd, Cilium node networking, and explicitly approved node-critical security/telemetry DaemonSets may execute on control-plane nodes. No Aerealith application, Istio waypoint, gateway, OpenBao, Prometheus, Flux controller, or general platform Deployment is scheduled there.

```
# Talos cluster-level intent (illustrative; validate schema for pinned Talos release)
cluster:
  allowSchedulingOnControlPlanes: false
  network:
    podSubnets:
      - 10.200.0.0/16
    serviceSubnets:
      - 10.201.0.0/16
  apiServer:
    admissionControl:
      - name: PodSecurity
        configuration:
          apiVersion: pod-security.admission.config.k8s.io/v1
```

```
kind: PodSecurityConfiguration
defaults:
  enforce: restricted
  enforce-version: latest
  audit: restricted
  audit-version: latest
  warn: restricted
  warn-version: latest
exemptions:
  namespaces: [kube-system]
  usernames: []
  runtimeClasses: []
```

Talos documents that Node + RBAC authorization, audit logging, secret encryption at rest, etcd mutual TLS, disabled anonymous authentication, kubelet certificate authentication, and restricted security defaults are available or enforced in the platform. [\[R2\]](#)

8. API, etcd, and node firewall policy

Service	Exposure	Required control
Talos API 50000/TCP	Per-node IP only	Management VPN/recovery subnet; mTLS; never public
Kubernetes API 6443/TCP	Talos VIP + node IPs	Workers, management VPN, approved collectors; OIDC/RBAC for humans
etcd 2379-2380/TCP	Control-plane node IPs only	Only CP /32 set; mTLS; never workers or admin LAN
Kubelet 10250/TCP	Node IPs	Control plane and approved monitoring only
Cilium VXLAN 8472/UDP	K8s nodes	Kubernetes node IP set only
Cilium WireGuard 51821/UDP	K8s nodes	Kubernetes node IP set only
Istio HBONE 15008/TCP	Worker nodes	Worker node set only; validate pinned release

Configure Talos ingress firewall with default block and explicit per-role rules. Talos implements these rules with nftables and recommends applying risky changes in try mode so a bad rule can revert instead of locking out the node. ^[R5]

```
apiVersion: v1alpha1
kind: NetworkDefaultActionConfig
ingress: block
---
apiVersion: v1alpha1
kind: NetworkRuleConfig
name: talos-api-admin
portSelector:
  ports: [50000]
  protocol: tcp
ingress:
  - subnet: 10.250.0.0/24
---
apiVersion: v1alpha1
kind: NetworkRuleConfig
name: cilium-vxlan
portSelector:
  ports: [8472]
  protocol: udp
ingress:
  - subnet: 10.42.0.0/24
```

Do not enable two host firewalls blindly

Talos ingress firewall owns node-service filtering. Keep Cilium Host Firewall disabled initially so nftables/BPF ordering cannot silently bypass or conflict with the OS rules. Cilium NetworkPolicy still owns pod traffic. Talos explicitly warns that another host firewall layer can take precedence. ^[R5]

9. Cilium network security plane

Cilium runs on every Kubernetes node, including control planes, because the API server and node-local proxy paths must be able to reach pods. The selected production profile retains kube-proxy because Cilium explicitly recommends that arrangement for the least disruptive Istio integration. Full kube-proxy replacement is supported, but it adds special socket-LB settings and is not worth the extra interaction risk in the first secure release. [\[R7\]](#)

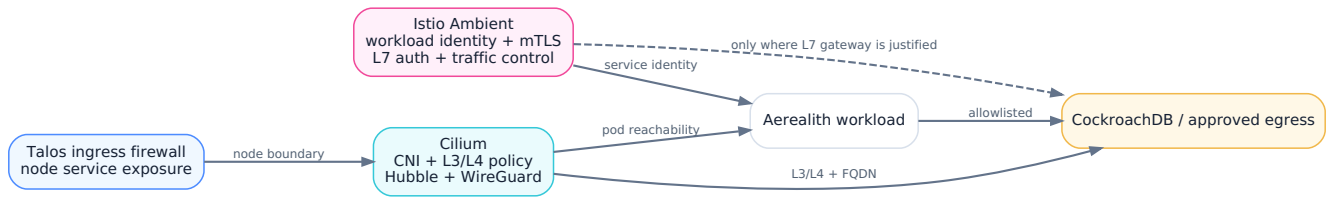
```
# Cilium values - intent baseline, pin and validate exact chart keys
kubeProxyReplacement: false
cni:
  exclusive: false
bpf:
  masquerade: false
routingMode: tunnel
tunnelProtocol: vxlan
encryption:
  enabled: true
  type: wireguard
  nodeEncryption: true
operator:
  replicas: 2
hubble:
  relay:
    enabled: true
  ui:
    enabled: false
hostFirewall:
  enabled: false
```

Cilium 1.20 is e2e-tested with Kubernetes 1.33 through 1.36 and lists Talos Linux as a supported host platform. [\[R8\]\[R35\]](#)

With Cilium under Istio Ambient, configure `cni.exclusive=false` and keep BPF masquerading disabled; Istio documents these as platform prerequisites. If default-deny policy is used, add the documented allowance for Ambient kubelet health probes sourced from 169.254.7.127/32. [\[R9\]](#)

```
apiVersion: cilium.io/v2
kind: CiliumClusterwideNetworkPolicy
metadata:
  name: allow-ambient-hostprobes
spec:
  enableDefaultDeny:
    ingress: false
    egress: false
  endpointSelector: {}
  ingress:
    - fromCIDR:
        - 169.254.7.127/32
```

10. Istio Ambient service security



Clear security ownership prevents Cilium and Istio from enforcing conflicting L7 policies.

Istio Ambient is selected instead of mandatory sidecars. Ztunnel provides node-level secure L4 transport and workload identity; waypoint proxies are introduced only where an application needs L7 authorization, traffic policy, or service-level canary routing. The control plane and Tier-0 infrastructure are not meshed by default.

Responsibility	Cilium	Istio
Pod connectivity	Owner	No
L3/L4 default deny	Owner	Supplemental identity checks
Node-to-node encryption	WireGuard	No
Workload identity	Kubernetes labels/identity context	Owner
Service-to-service mTLS	No	Owner
HTTP method/path authorization	Avoid in meshed apps	Owner via waypoint/gateway
Ingress traffic shifting	No	Owner
FQDN egress	Owner by default	Only for special L7 egress cases
Flow visibility	Hubble	Request/service telemetry

Istio 1.30 supports Kubernetes 1.32-1.36, and the current 1.30 patch line received security updates through 1.30.4 on 27 August 2026. Production should track supported patch releases rather than pinning a stale minor. [\[R10\]](#)[\[R11\]](#)

```

apiVersion: security.istio.io/v1
kind: PeerAuthentication
metadata:
  name: strict-mtls
  namespace: aerealith-core
spec:
  mtls:
    mode: STRICT
  
```

Mesh enrollment rule

Mesh only namespaces that host Aerealith product workloads and explicitly selected shared services. Do not initially mesh kube-system, flux-system, openbao, kyverno, or the control plane. Circular dependencies are a security and recovery problem, not a feature.

11. Namespaces, node pools, and security zones

Namespace	Node pool	PSA	Mesh	Purpose
kube-system	control-plane + all nodes	Privileged exceptions	No	Kubernetes, Cilium, CoreDNS
istio-system	platform	Restricted + narrow exceptions	No	Istiod, CNI management
istio-ingress	platform	Restricted	Ambient/gateway	Private ingress gateway
flux-system	platform	Restricted	No	GitOps controllers
kyverno	platform	Restricted + controller exceptions	No	Admission policy
cert-manager	platform	Restricted	No	Certificate lifecycle
openbao	platform	Restricted	No	Runtime secrets / PKI
observability	platform	Restricted	Optional	Prometheus, OTel, Loki, Jaeger, Grafana
security	platform	Privileged exceptions documented	No	Falco, Kubescape, Trivy Operator
aerealith-edge	application	Restricted	Ambient	Frontend/BFF
aerealith-core	application	Restricted	Ambient	API/auth/entity/memory
aerealith-async	application	Restricted	Ambient	Workers/jobs
aerealith-integrations	application	Restricted	Ambient	Discord/integrations
aerealith-ai	application/GPU	Restricted	Ambient	AI services/jobs
evidence-system	platform	Restricted	No	Read-only collectors and evidence export

```

apiVersion: v1
kind: Namespace
metadata:
  name: aerealith-core
  labels:
    pod-security.kubernetes.io/enforce: restricted
    pod-security.kubernetes.io/audit: restricted
    pod-security.kubernetes.io/warn: restricted
    istio.io/dataplane-mode: ambient
    aerealith.io/owner: platform
    aerealith.io/data-class: confidential

```

Namespace creation should be admission-controlled: no namespace is production-ready until it has an owner, data classification, Pod Security labels, ResourceQuota/LimitRange where appropriate, default-deny network policy, and a documented mesh decision.

12. Default-deny network policy model

Every workload namespace starts with deny-all ingress and egress. Allow rules are written in the same pull request as the consuming workload. Network location is not identity, so Cilium allow rules are a reachability control; Istio/OpenBao/database identities still authorize the operation.

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: default-deny
  namespace: aerealith-core
spec:
  podSelector: {}
  policyTypes: [Ingress, Egress]
```

Source identity	Allowed destination	Reason
cloudflared	Istio Gateway only	Public origin path
Istio Gateway	edge/core routes declared by HTTPRoute	No broad namespace wildcard
frontend/BFF	API only	No direct DB or secret store
core service SA	Approved CRDB/OpenBao/Valkey endpoints	Least-service reachability
async worker SA	Queue/CRDB/object store + exact external APIs	Job-specific
Prometheus	metrics endpoints only	Read-only scrape
OTel collectors	log/trace receivers	Telemetry only
Flux	Git/OCI/DNS only	No arbitrary Internet
all other	none	Default deny

For external IPs such as CockroachDB VMs, use Cilium policies scoped to service accounts and exact /32 destinations. The CockroachDB host firewall separately restricts source addresses to Kubernetes worker IPs and management VPN peers. This two-sided model is essential on the flat LAN.

13. Identity, RBAC, and administrative access

Human and machine identities are separated. Talos API certificates are infrastructure-root credentials; Kubernetes user identities are not reused for Talos. Routine Kubernetes administration should use OIDC with MFA when the self-hosted identity provider is ready. Until then, use named, short-lived administrator kubeconfigs and keep the static cluster-admin credential offline.

Principal	Authentication	Authorization	Review
Talos operator	Talos mTLS certificate + VPN	Talos role certificate	Quarterly and every staff change
Kubernetes admin	OIDC + FIDO2/MFA	RBAC groups; no daily cluster-admin	Quarterly
Break-glass	Offline certificate	Time-bounded cluster-admin	Every use; rotate afterward
Flux	ServiceAccount	Only reconciled API groups/namespaces	Each controller upgrade
Aerealith services	Dedicated SA + Istio identity	RBAC only when API access is needed	Per release + quarterly
Prometheus/Kubescape	Dedicated SA	Read-only discovery; no Secret reads	Quarterly
Evidence collector	Dedicated SA	Read-only source + write-only archive credential	Monthly diff

- Set `automountServiceAccountToken: false` by default.
- Use projected bound service-account tokens with explicit audience when Kubernetes API access is required.
- Do not grant wildcard verbs/resources to application namespaces.
- Do not bind application teams to cluster-admin for troubleshooting.
- Generate an access export and reviewer sign-off every quarter.
- Protect GitHub, Cloudflare, Proxmox, OpenBao, and backup accounts with hardware-backed MFA where supported.

14. Pod Security and Kyverno admission policy

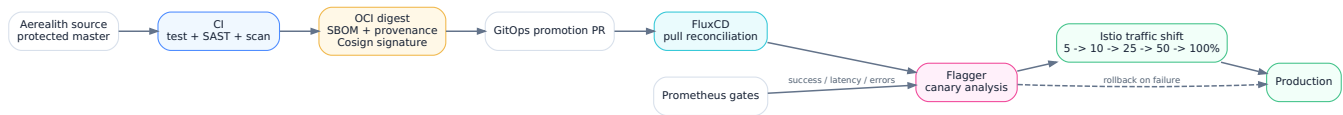
Pod Security Admission provides the non-negotiable baseline; Kyverno adds organizational policy, image verification, required metadata, resilience requirements, and exception governance. Kyverno 1.19 has feature parity in its CEL-based policy family and deprecates legacy Policy/ClusterPolicy APIs, so new work should use the newer policy types. ^[R14]

Control	Production rule	Exception
Privileged	Deny	Only named system components with expiry
Root	runAsNonRoot=true	Image must be rebuilt
Privilege escalation	false	Rare system-only exception
Capabilities	Drop ALL	Add minimal named capability
Seccomp	RuntimeDefault	No unconfined application pods
Read-only root FS	Required where supported	Document write path and compensating mount
Host namespaces	Deny hostPID/hostIPC/hostNetwork	System DaemonSet only
hostPath	Deny arbitrary	Exact path + owner + expiry
Images	Digest required in production	None
Registry	Approved registries only	Security approval
Signature	Cosign verification	Emergency exception with incident/change ticket
Resources	Requests/limits required	Calibrated system DaemonSet
Resilience	Critical service >=3 replicas + PDB + spread	Approved service-tier exception

Introduce policies in Audit first, remediate violations, then move to Enforce. Policy-engine availability is monitored; an admission controller that is silently failing open is a security incident.

15. Software supply-chain security

The production cluster should consume immutable, verifiable artifacts. CI is a builder and evidence producer, not a deployment authority.



Build once, promote by digest, and use Flagger/Istio to prove the release before full traffic.

Stage	Required control
Source	Protected master; issue/PR; CODEOWNERS; signed or verified commits; dependency lockfiles
CI runner	Ephemeral where possible; minimal token permissions; pinned Actions by commit SHA
Build	Multi-stage/rootless image where possible; deterministic lockfile; no secrets in layers
Scan	Trivy filesystem/config/dependency/final-image scan; secret scan
SBOM	SPDX or CycloneDX attached to digest
Provenance	SLSA-style build provenance/attestation where supported
Sign	Cosign signature bound to immutable digest
Registry	Approved registry; retention; audit; pull-only runtime credentials
Admission	Kyverno verifies digest + signature + identity
Promotion	GitOps PR changes only tested digest and related config

Do not enable AlwaysPullImages automatically

For this on-prem HA design, forcing every restart to contact the registry can reduce availability during a registry outage. Pin by digest, verify signatures at admission, use immutable retention, and keep node image caches. Evaluate AlwaysPullImages only if the multi-tenant cache threat outweighs registry-dependency risk.

16. Self-hosted OCI registry strategy

Because the broader goal is self-hosting, the preferred long-term registry is Harbor, a CNCF Graduated project that provides registry RBAC, scanning, signing integrations, replication, and audit features. [\[R17\]](#)

Do not make the first production cluster depend on an over-engineered Harbor HA stack before the cluster itself is recoverable. Use a staged registry strategy:

1. Bootstrap: keep GHCR or another existing registry as an explicit external development-plane exception while the cluster is built.
2. Self-hosted mirror: deploy Harbor after storage, secrets, and backups are working; mirror all production images and signatures locally.
3. Production cutover: switch production references to Harbor only after Harbor restore and outage behavior are tested.
4. Recovery: export the minimum Tier-0/Tier-1 OCI image set to offline/PBS storage so an empty cluster can be rebuilt even if the registry database is unavailable.

Harbor concern	Design response
Registry availability	Not in request path, but required for reschedule/scale; monitor and cache
Database/Redis dependencies	Design separately; do not pretend a single embedded DB is HA
Blob storage	Use durable S3-compatible storage or backed persistent storage
Scanning	Trivy integration; CI scan remains independent
Trust	Cosign signatures verified by cluster, not by registry reputation alone
Backups	Config, database, blobs, signing metadata; quarterly restore

17. Secrets management and PKI

Use three separate secret layers with different recovery characteristics: offline root/recovery material, SOPS + age for bootstrap, and OpenBao for runtime secrets and internal PKI. cert-manager, now CNCF Graduated, automates namespaced certificate lifecycle. [\[R18\]](#)

Secret class	Authority	Examples
Offline recovery	Encrypted physical custody	Talos CA recovery material, OpenBao unseal/recovery, root CA, production age recovery
Bootstrap	SOPS + age	Flux bootstrap, initial OpenBao config, tunnel seed credentials
Runtime dynamic	OpenBao	Database users, API tokens, short-lived credentials
Service certificates	OpenBao intermediate + cert-manager	Internal TLS, ingress origin, monitoring client certs
Mesh identity	Istio CA anchored to approved trust chain	SPIFFE-style workload identities and mTLS

The production age private key used by Flux is not the same key that protects Talos machine configuration. Talos machine configs can contain cluster CA and node secret material; their encryption/decryption path should live outside Kubernetes so compromise of Flux cannot become compromise of the node-management root.

- OpenBao: 3 Raft replicas, one per platform worker/PVE zone; PDB minAvailable=2.
- OpenBao is not initially meshed; use its own TLS/auth boundary and Cilium reachability controls.
- Use Kubernetes auth roles bound to namespace + service account + audience.
- Use short certificate lifetimes and automatic renewal; alert on 30/14/7-day expiry for long-lived CA/intermediate assets.
- Keep root CA offline. OpenBao holds only an intermediate used for online issuance.
- Record every secret-engine, auth-method, policy, root/recovery, and seal operation as high-priority audit telemetry.

18. Canary and progressive delivery

Flagger is selected because it matches the Flux operating model and integrates directly with Istio and Prometheus. A canary is a deployment safety control, not merely a release convenience. ^[R13]

Phase	Traffic	Required evidence
Initialize	0%	Candidate pods healthy; config validation passes
Step 1	5%	Success rate, latency, dependency errors, saturation
Step 2	10%	Same + business correctness metrics
Step 3	25%	No security/audit regression; no queue/backlog growth
Step 4	50%	Sustained analysis window
Promote	100%	All automated gates pass; manual approval if high-risk
Rollback	0% candidate	Automatic if threshold exceeded; incident/change event retained

```
apiVersion: flagger.app/v1beta1
kind: Canary
metadata:
  name: aerealith-api
  namespace: aerealith-core
spec:
  targetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: aerealith-api
  progressDeadlineSeconds: 600
  analysis:
    interval: 1m
    threshold: 5
    maxWeight: 50
    stepWeight: 5
    metrics:
      - name: request-success-rate
        thresholdRange: { min: 99.0 }
        interval: 1m
      - name: request-duration
        thresholdRange: { max: 750 }
        interval: 1m
```

Database migration rule

Automatic traffic rollback cannot undo a destructive schema change. Production migrations must follow expand -> deploy compatible code -> backfill/migrate -> verify -> contract in a later release.

19. Flux GitOps operating model

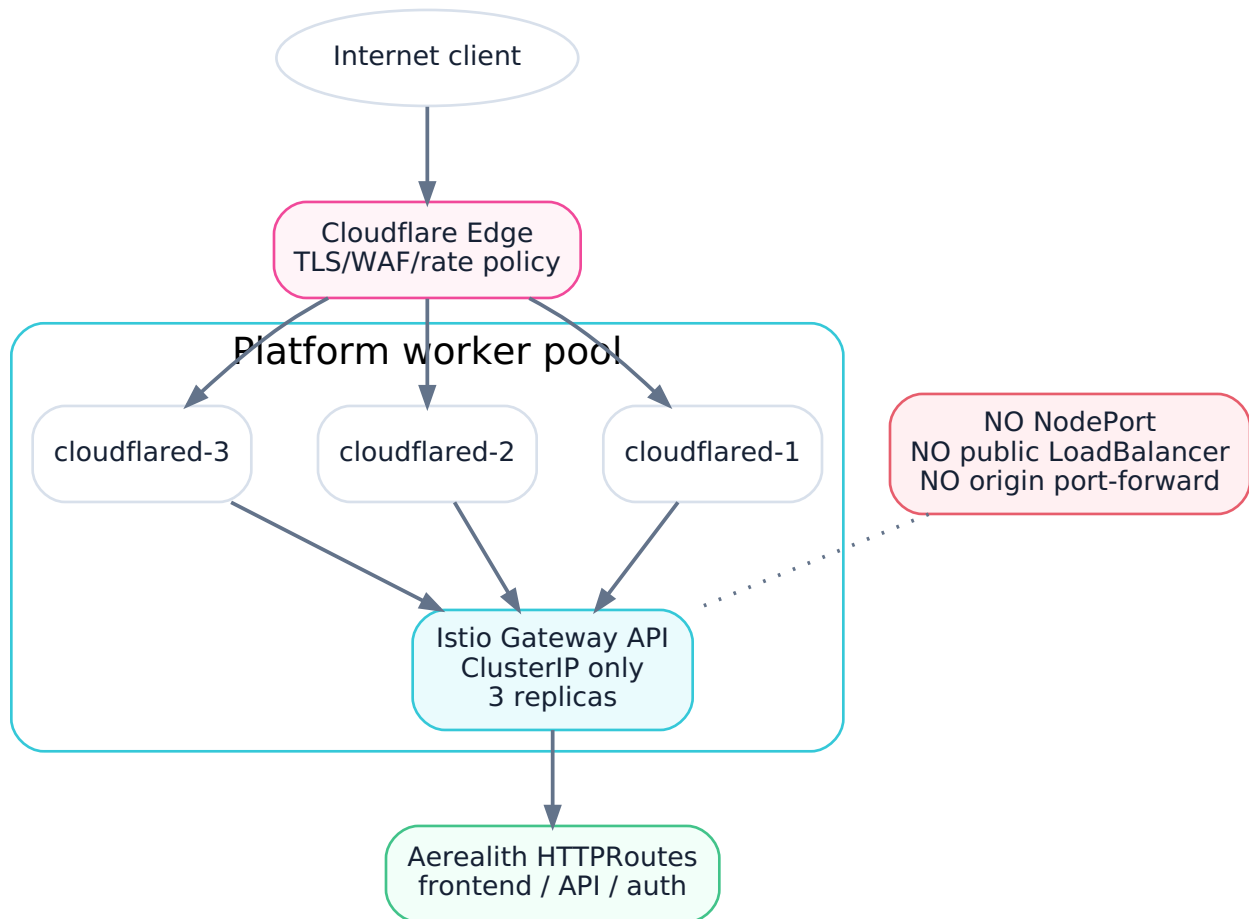
Flux remains the sole production reconciler. CI must not hold production kubeconfig or Talos credentials. Flux supports SOPS decryption, which keeps encrypted bootstrap secrets in Git while preserving pull-based deployment. [\[R12\]](#)

```
aerealith-gitops/
  clusters/
    homelab/
    staging/
    production/
  infrastructure/
    cilium/
    istio/
    flux/
    kyverno/
    cert-manager/
    openbao/
    observability/
    security/
    cloudflared/
    flagger/
  applications/
    aerealith/
      base/
      overlays/{homelab,staging,production}/
  policies/
  secrets/
  docs/
```

Production changes use short-lived `infra/*`, `deploy/*`, `promote/*`, `security/*`, `fix/*`, and `chore/*` branches and merge into one protected master branch. Environments are directories, not permanent branches.

Reconciliation order	Contents
00-bootstrap	CRDs and sources
10-network	Cilium adoption/bootstrap state
20-policy	PSA labels, Kyverno, default-deny baselines
30-secrets-pki	OpenBao, cert-manager issuers
40-mesh-ingress	Istio, gateway, cloudflared
50-observability-security	Prometheus, OTel, Falco, Kubescape, Trivy
60-data-adapters	External CRDB endpoints, caches/object stores
70-applications	Aerealith workloads
80-progressive-delivery	Flagger canaries and analysis templates
90-evidence	Exports, policy reports, control evidence

20. Cloudflare Tunnel and private ingress



Public traffic has no direct path to a node listener. Cloudflare connectors initiate outbound tunnels and reach a private ClusterIP Istio Gateway.

Cloudflare Tunnel remains the explicit external ingress exception. Three cloudflared replicas run on the three platform workers, with topology spread by Proxmox zone and hostname plus a PDB of 2. Cloudflare documents replica connectors across different hosts as the HA pattern. ^[R30]

- One named tunnel with three connectors - not three unrelated tunnels.
- Outbound TCP/UDP 7844 only, plus exact DNS/HTTPS control endpoints.
- Private Istio Gateway Service is ClusterIP only.
- No NodePort, no LoadBalancer, no hostPort, no WAN 80/443 port forward to the cluster.
- cloudflared verifies the internal origin certificate; do not use noTLSVerify.
- Management services are never added as public hostnames.
- If ePHI traverses Cloudflare, complete the exact plan/service BAA and subprocessor review before production.

Cloudflare does not create internal zero trust

Tunnel transport protects the edge connection. It does not replace Istio identity, application authentication, object-level authorization, database TLS, or audit logging.

21. Runtime security and posture management

The security stack is layered but intentionally not duplicated. Static posture, vulnerability management, runtime detection, network telemetry, and audit each have a distinct job.

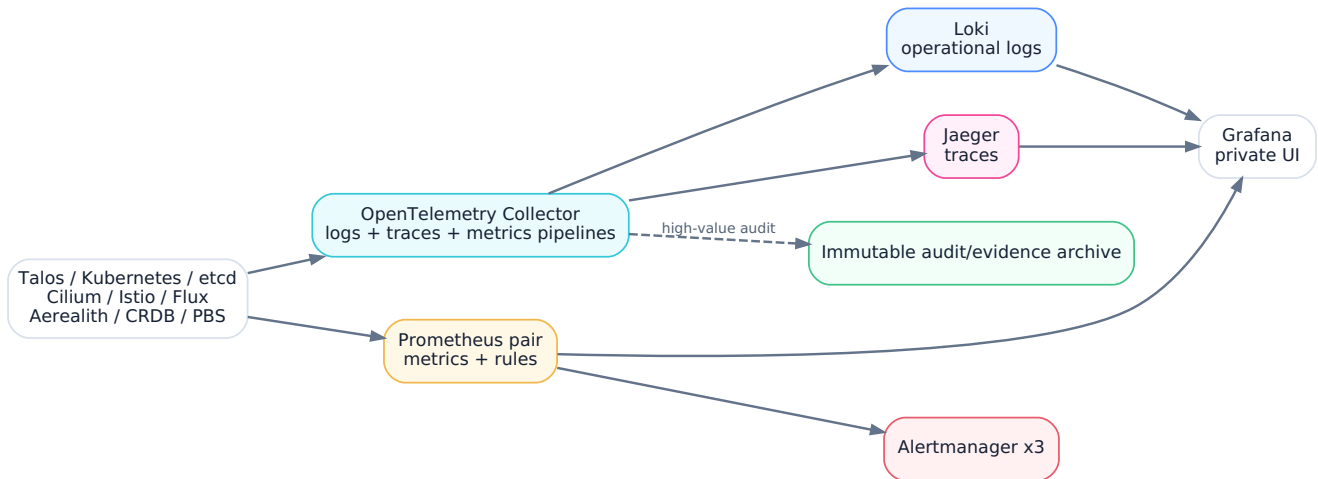
Tool	Maturity / role	Decision
Cilium/Hubble	CNCF Graduated; network policy/flow visibility	Required
Istio	CNCF Graduated; mesh identity/mTLS/L7	Required
Kyverno	CNCF project; admission/image policy	Required
Trivy / Trivy Operator	CVE, config, IaC, image scanning	Required
Kubescape	CNCF Incubating; posture/compliance scanning	Required
Falco	CNCF Graduated; runtime syscall/event detection	Required after tuning
Tetragon	eBPF runtime observability/enforcement; Talos supported with specific mounts	Optional advanced phase
OPA Gatekeeper	Second policy engine	Not initially - avoid duplicate admission logic

Kubescape is a CNCF Incubating project designed to scan clusters, manifests, registries, and images against security and compliance frameworks such as CIS, NSA-CISA, and MITRE ATT&CK.^[R16]

Falco should be deployed with the Falco Operator, initially in observation mode. Its privileged host access is a documented Kyverno exception, and alerts are tuned for the Talos/containerd environment before paging.^[R15]

Tetragon is optional because it overlaps some runtime/security observability. Its Kubernetes documentation includes specific Talos Linux mount requirements, which makes it practical if deeper eBPF tracing or enforcement is later justified.^[R36]

22. Observability and telemetry architecture



OpenTelemetry standardizes telemetry pipelines; Prometheus remains the metrics and alert-rule engine.

Use OpenTelemetry Collector as the default telemetry transport instead of tying applications directly to a vendor-specific agent. OpenTelemetry graduated in CNCF in 2026 and provides standard APIs, SDKs, and collectors for metrics, logs, and traces. [\[R19\]](#)

Component	Deployment	HA / retention intent
Prometheus	2 independent replicas on platform workers	7-15 day local retention; no shared TSDB
Alertmanager	3 replicas	HA mesh; minAvailable=2
OpenTelemetry Collector	DaemonSet + 2 gateway collectors	Buffer/backpressure limits; OTLP mTLS internally where useful
Loki	3 replicas or small HA topology	Operational logs; retention by data class
Jaeger	2+ collector/query components	Trace backend; sampling limits
Grafana	2 replicas if UI HA is required	Private via VPN; dashboards provisioned from Git
Hubble Relay	2 replicas	Network flow visibility; UI disabled/private
External Prometheus 10.42.0.20	Outside-cluster sentinel	Proxmox, PBS, network, synthetic public checks

Talos can forward service and kernel logs to remote TCP/UDP JSON-line destinations; use that path to preserve host logs even when the Kubernetes API is unavailable. [\[R32\]](#)

23. Audit logging and evidence-quality telemetry

Audit data must answer different questions at different layers. Kubernetes API audit proves administrative actions against cluster resources; Talos logs prove node lifecycle/configuration events; OpenBao audit proves secret operations; Aerealith application audit proves which user touched which protected business object.

Source	Minimum events	Retention handling
Kubernetes API	authn failures, RBAC changes, Secret metadata access, exec/port-forward, admission policy changes, deployments	Searchable + immutable high-value export
Talos	config apply, upgrade, reboot, API authentication, service/kernel events	Central collector + evidence snapshots
Cilium/Hubble	denies, unusual east-west, DNS/egress anomalies	Short operational + incident export
Istio	mTLS failures, authz denies, gateway access	Operational; redact sensitive headers
OpenBao	seal/unseal, root, auth/policy, secret lease, denied access	Restricted immutable audit path
CockroachDB	node/SQL security events, privileged SQL, backup operations	Data-class retention
Flux/Kyverno/Flagger	reconciles, drift, admission denies, exceptions, rollout/rollback	Per change + monthly evidence
Aerealith	actor, tenant, object, action, purpose, result, correlation ID	Purpose-specific protected audit store

Do not log secrets or raw protected payloads

Authorization headers, session cookies, passwords, tokens, private keys, raw prompts containing sensitive data, and full database rows must not appear in ordinary telemetry. Audit evidence is still regulated data if it contains regulated identifiers.

24. CockroachDB security and placement

CockroachDB remains outside Kubernetes on three dedicated VMs, one per physical Proxmox host. This is still the right design: the primary database must not require a functioning Kubernetes storage plane in order to recover.

Node	Locality	Security baseline
crdb-01	region=idaho,zone=pve-a	TLS node cert; encrypted disk; host firewall; dedicated service account
crdb-02	region=idaho,zone=pve-b	Same
crdb-03	region=idaho,zone=pve-c	Same

CockroachDB secure deployments rely on node and client certificates with correct SANs. Use an OpenBao-issued intermediate after staging compatibility, or CockroachDB-native certificate tooling if the operational risk of integrating the PKI is higher. [\[R23\]](#)

- Multi-host connection string lists all three private nodes; do not add a single HAProxy SPOF.
- Separate runtime, migration, monitoring, backup, analytics, and break-glass identities.
- Host firewall allows 26257 only from CRDB peers, approved Kubernetes worker addresses, and named management peers.
- Admin UI/metrics port 8080 is never public; restrict to monitoring and VPN.
- Use native scheduled backups as authoritative DB backups; monitor completion and restore regularly. [\[R24\]](#)
- A one-node loss must not require manual data repair before application service resumes.

25. Storage and persistence strategy

Keep the storage design simple. Etcd, CockroachDB, and OpenBao already provide application-level replication. Adding Ceph or Longhorn underneath every stateful system multiplies write amplification and failure modes without adding a new physical failure domain.

Storage layer	Decision	Why
Proxmox local mirrored ZFS	Selected	Device redundancy, checksums, snapshots, clear host boundary
Talos local user volumes	Selected for platform state that is already replicated	Predictable local performance
Ceph	Not initially	Three-node maintenance margin and operational complexity are poor
Longhorn	Not initially	Duplicates synchronous replication and may conflict with Talos noexec/immutable assumptions
Single NFS/NAS	Not Tier-0	Creates another storage SPOF unless independently HA
Object storage	Use only for workloads that benefit from S3 semantics	Must still have offsite backup

OpenBao Raft, Prometheus local TSDB, and other local-PV components must be deliberately placed on one replica per physical host. A local PV is not a backup and not a migration mechanism.

26. Backup and disaster recovery



Backups use native application snapshots plus a physically independent Proxmox Backup Server and offsite/offline copy.

Talos documents consistent etcd snapshots through `talosctl etcd snapshot` and full control-plane recovery from a snapshot. Machine configuration must also be recoverable, because replacing nodes requires the same cluster secret material. [\[R6\]](#)

Asset	Authoritative backup	Initial target	Restore proof
etcd	talosctl etcd snapshot	Every 4-6h + before control-plane change	Quarterly empty-cluster recovery
Talos machine configs/secrets	Encrypted recovery bundle	Every approved change	Quarterly node rebuild
CockroachDB	Native scheduled backup	<=15m incremental target + daily full strategy	Monthly isolated DB restore
OpenBao	Raft snapshot	Every 4-6h + before upgrade/rekey	Quarterly new-cluster restore/unseal
Proxmox VMs	PBS	Nightly	Monthly representative VM restore
GitOps repositories	Git mirror + encrypted offline bundle	Every merge	Quarterly bare-cluster bootstrap
Audit/evidence	Append-only immutable export	Hourly/daily by class	Monthly hash/query sample
PKI/recovery material	Encrypted offline custody	After every key hierarchy change	Quarterly recovery ceremony

PBS should be physical hardware outside the three-node Proxmox cluster. Proxmox Backup Server provides encryption, integrity verification, deduplication, access control, and remote sync features; its restore capabilities are only valuable if they are actually exercised. [\[R31\]](#)

27. Failure-mode analysis

Failure	Expected automatic result	Manual action	Residual risk
One control-plane VM	K8s API continues via 2/3 etcd; VIP re-elects	Rebuild member on same physical domain	Second CP loss
One platform worker	Istio/Flux/security replicas continue on two	Repair/rebuild; local-PV member rejoin	Capacity and quorum placement
One application worker	Critical app replicas continue on two	Repair/rebuild; queues retry	N+1 request capacity
One entire Proxmox host	2 CP + 2 sys + 2 app + 2 CRDB remain	Stabilize; rebuild one member of each set	Shared core/gateway/ISP
One cloudflared	Other connectors remain	Investigate connector	Cloudflare account/provider
One Istio gateway/waypoint	Replica routing continues	Replace pod; inspect config	Insufficient replicas
One CRDB VM	2/3 database quorum	Repair/rebalance	Second DB loss
One OpenBao replica	2/3 Raft remains	Replace/rejoin	Second replica loss
USG-Pro-4	Internet/VPN lost	Replace/cut over gateway	Current edge SPOF
USW Aggregation	Cluster hosts isolated	Replace switch	Current core SPOF
Site power/cooling	After UPS runtime, primary site unavailable	Invoke site recovery	Single-site compute
Bad GitOps commit	Policy/canary may block, but some bad state can reconcile	Revert + incident review	Git trust plane

Availability ceiling

Kubernetes can be HA while the site is not. The existing single gateway, aggregation switch, ISP, power/cooling domain, and site remain larger availability limits than the control plane.

28. Incident containment and recovery



Incident workflow preserves evidence first, then contains with identity/network controls and rebuilds immutable components.

1. Declare severity and incident commander; synchronize clocks and preserve evidence.
2. Freeze risky automated changes; identify whether Flux must be temporarily suspended for affected resources.
3. Revoke exposed credentials and deny network paths with Cilium/Talos firewall policies.
4. Snapshot relevant audit/log data before destructive remediation.
5. Rebuild compromised Talos node or workload from known-good image/config rather than hand-repairing it.
6. Restore data only from verified backups; rotate secrets/PKI as required.
7. Re-enable traffic through canary/progressive delivery, not instant full cutover.
8. Close with root cause, control failure, evidence links, owner, due date, and risk acceptance if remediation is deferred.

Break-glass reconciliation

If an emergency requires direct `kubectl` mutation, record the exact diff and incident ID, then reconcile Git immediately. The cluster must not remain in a permanently undocumented state.

29. Patch, upgrade, and vulnerability management

Security depends on staying within supported release windows. Talos supports only recent minor releases for community security updates, while Cilium and Istio support specific Kubernetes ranges. Treat the version matrix as an operational control, not a one-time installation choice.

Component	Production rule
Talos	Latest supported patch on an accepted minor; control-plane one node at a time; workers one failure domain at a time
Kubernetes	Upgrade one minor at a time; staging compatibility test first
Cilium	Latest patch before minor upgrade; run preflight/connectivity tests; never skip unsupported minor paths
Istio	Supported minor only; current patch; revision-based upgrade/canary where possible
Kyverno	Policy test suite passes before controller/API migration
Flux/Flagger	Pin controller/chart versions; reconcile staging before production
OpenBao/CRDB	Native backup before upgrade; one quorum member at a time
Talos Secure Boot image	Preserve signing/PCR keys and validate rollback before fleet rollout

As of 30 August 2026, an example compatibility baseline is Talos 1.13.9 / Kubernetes 1.36.3, Cilium 1.20.x, and Istio 1.30.4. Do not treat this document as a permanent pin: verify the current stable Talos release and every project compatibility matrix at implementation time. Cilium documents Kubernetes 1.33-1.36 compatibility for the 1.20 line, and Istio 1.30 supports Kubernetes 1.32-1.36. [\[R8\]](#)[\[R10\]](#)[\[R11\]](#)

30. Vulnerability-management SLA

Severity	Target remediation	Production rule
Critical - exploited or Internet-reachable	24 hours	Emergency change/canary; isolation if patch unavailable
Critical - no known exploitation	72 hours	Patch or documented compensating control
High	7 days	Patch in maintenance window; risk exception required if missed
Medium	30 days	Planned remediation
Low	90 days / backlog	Risk-based prioritization

- Scan code and images on every build.
- Rescan running production images daily or on vulnerability-database update.
- Generate a daily cluster inventory of image digests and node versions.
- Track vulnerability age, not just raw count.
- Require exception owner, justification, compensating control, approval, and expiry.
- Patch Talos/Proxmox/USG/firmware as separate asset classes with their own support windows.
- Record emergency mitigations such as NetworkPolicy or feature disablement as temporary controls, not permanent fixes.

31. Security standards and compliance mapping

Technical architecture supports control objectives, but auditors certify or attest an organization and system of controls, not a list of tools. ISO/IEC 27001 is an ISMS standard; SOC 2 is an independent CPA attestation; HIPAA requires administrative, physical, and technical safeguards; GDPR requires risk-appropriate security and accountability. [\[R26\]](#)[\[R27\]](#)[\[R28\]](#)[\[R29\]](#)

Control domain	Technical implementation	Evidence examples	Framework relevance
Access	VPN, Talos mTLS, OIDC/RBAC, OpenBao policy	RBAC export, admin cert inventory, quarterly review	ISO, SOC 2, HIPAA, GDPR
Change/configuration	Protected Git, Flux, Talos config review	PR approvals, config hashes, reconciliation records	ISO, SOC 2, HIPAA
Network security	Talos firewall, Cilium, Istio mTLS	Firewall exports, NetworkPolicy, Hubble, cert inventory	All
Supply chain	Trivy, SBOM, Cosign, Kyverno	Scan reports, SBOM, signature verification	ISO, SOC 2
Logging/detection	Audit, Falco, Hubble, application audit	Alert tests, immutable audit manifests, incidents	All
Resilience	HA, backups, restore tests	Restore reports, failover timings, backup checksums	ISO, SOC 2 Availability, HIPAA
Privacy lifecycle	Classification, retention, deletion controls	RoPA/DPIA/deletion tests	ISO 27701/GDPR
Physical/media	Rack, UPS, disk custody, sanitization	Access logs, UPS tests, asset and disposal records	ISO, HIPAA
Suppliers	Cloudflare/GitHub/provider review	BAA/DPA/subprocessor and risk records	ISO, SOC 2, HIPAA, GDPR

32. Automated compliance evidence

Every important automated control should have an owner, collection method, frequency, retention class, and review step. A dashboard is not evidence unless its underlying state is exported, attributable, and retained.

```
compliance/
  controls/
  mappings/
    iso27001-2022.yaml
    soc2-tsc.yaml
    hipaa.yaml
    gdpr.yaml
  evidence/
    access/
    network/
    policy/
    supply-chain/
    backups/
    availability/
    physical/
  policies/
  risk/
  incidents/
  privacy/
  vendors/
  audits/
```

Evidence family	Automated artifact	Review cadence
Talos/node	Machine config hash, node version, firewall rules, secure-boot/encryption status	Daily snapshot; monthly review
Kubernetes access	RBAC, bindings, admin auth settings	Monthly diff; quarterly sign-off
Network	Cilium policies, Talos firewall docs, public route scan	Daily; monthly review
Admission	Kyverno policies, PolicyReports, exceptions	Daily; weekly exception review
Supply chain	Digest, Trivy, SBOM, Cosign verification	Per release + daily rescan
GitOps/canary	Flux revision, Flagger analysis, rollback/promotion	Per change
Backups	Job, checksum, retention, restore result	Per job; monthly/quarterly review
Detection	Audit health, Falco rules, alert test results	Continuous + monthly test
Physical	UPS/sensors/assets/media custody	Monthly/quarterly

Evidence minimization

Do not copy raw PHI/customer payloads into the compliance repository. Store sensitive evidence in a restricted immutable object store and commit only the manifest, hash, time window, control ID, and reviewer result.

33. Phased implementation plan

Phase	Outcome	Hard exit gate
0 - Inventory	Exact hosts, IPs, NICs, disks, power, routes, data classes	No invented IP/capacity values
1 - Proxmox foundation	Patched hosts, ZFS, VM firmware, firewall templates, PBS console path	Quorum + backup console tested
2 - Talos control plane	3 CP nodes + etcd + VIP + default-block host firewall	Kill one CP; API/etcd remain healthy
3 - Platform workers	3 sys workers with topology labels/taints	Loss of one sys worker does not remove critical platform
4 - Application workers	3 app workers + N+1 capacity proof	Loss of one app worker within SLO
5 - Cilium	CNI, WireGuard, Hubble, default-deny baseline	Cilium connectivity test + encryption proof
6 - Istio Ambient	istiod/ztunnel/gateway, strict mTLS in app namespaces	mTLS/authz test suite passes
7 - Flux/SOPS/Kyverno	GitOps authoritative, policies enforced	Direct deploy path disabled/documented
8 - Security stack	Trivy/Kubescape/Falco/OTel/audit	Detection and alert tests succeed
9 - Secrets/PKI	OpenBao/cert-manager/rotation	Raft snapshot restore succeeds
10 - Ingress/canary	cloudflared x3 + Flagger	Kill connector + force failed canary rollback
11 - Data/backups	CRDB + PBS + offsite/offline copy	DB/PBS/etcd restore proofs
12 - Aerealith production	Digest-pinned workloads and SLOs	One-host failure exercise succeeds
13 - Evidence/DR	Automated evidence + quarterly runbooks	Signed review and production readiness approval

Do not install the entire toolchain first and harden it afterward. Each phase closes with a failure test and evidence package before the next phase begins.

34. Production validation checklist

Cluster foundation

- Three control-plane nodes are on three different physical hosts.
- No normal workload can schedule on control-plane nodes.
- Talos API is reachable only from approved management sources.
- Kubernetes API VIP fails over and is not public.
- etcd quorum, snapshot, and restore have been tested.
- Talos ingress firewall default action is block on all cluster nodes.

Network and mesh

- Cilium connectivity test passes.
- WireGuard encryption state is verified cross-node.
- Default-deny policy exists in every production application namespace.
- Ambient host-probe exception is present if required.
- Istio strict mTLS is verified for selected namespaces.
- Cilium and Istio L7 policy responsibilities do not overlap.
- No public NodePort/LoadBalancer/hostPort exists.

Workload security

- PSA Restricted enforced on application namespaces.
- Kyverno policy suite is Enforce with documented exceptions only.
- All production images are digest pinned and signed.
- Trivy/Kubescape scans have no unaccepted Critical/High finding.
- Falco alerts reach the incident channel.
- ServiceAccounts are dedicated and token automount is off unless needed.

Availability and recovery

- Critical application and platform replicas are spread by zone and hostname.
- PDBs protect voluntary disruption.
- Two surviving workers of each pool have capacity for critical requests.
- Cloudflare connector, Istio gateway, OpenBao, and CRDB single-member failure tests pass.
- PBS restore, CRDB restore, OpenBao restore, and etcd recovery are documented and measured.

35. Definitive architecture

The architecture I would build

Talos Linux + upstream Kubernetes on three dedicated control-plane VMs, three platform workers, and three application workers. Use the Talos built-in layer-2 API VIP, Cilium on every Kubernetes node, and no ordinary workloads on the control plane. Keep kube-proxy for the initial Cilium + Istio integration. Mesh only application namespaces with Istio Ambient, enforce strict mTLS, and use waypoints only where L7 authorization or internal traffic management needs them.

Run Flux as the only production reconciler; SOPS for bootstrap; OpenBao/cert-manager for runtime secrets and PKI; Kyverno for admission; Trivy, Kubescape, and Falco for security; OpenTelemetry + Prometheus/Loki/Jaeger/Grafana for observability; Flagger + Istio for canary promotion; CockroachDB on three dedicated VMs; and a physical PBS with offsite/offline recovery.

The security model compensates for the intentionally flat LAN by making every destination enforce its own boundary: Talos ingress firewall for node services, Proxmox/guest firewalls for non-Kubernetes VMs, Cilium for pod reachability, Istio for service identity and L7 authorization, and application/database identities for the actual data operation. This is stronger than the v2 design because the host OS, node firewall, platform/app worker split, artifact trust, runtime detection, and compliance evidence are all treated as first-class architecture.

Keep	Replace / change
Three physical Proxmox failure domains	K3s -> Talos Linux + upstream Kubernetes
Cilium	kube-vip -> Talos built-in L2 VIP
Istio Ambient	Combined workers -> preferred platform/app worker split
Flux + Flagger	Generic host nftables -> Talos ingress firewall on K8s nodes
Cloudflare Tunnel	Vendor-specific telemetry agent -> OpenTelemetry Collector
CockroachDB outside K8s	Ad-hoc security scans -> Kubescape + Trivy + Falco evidence
OpenBao/cert-manager	Unbranded/plain PDF -> Aerealith-branded implementation bible

A. Configuration baselines

Control-plane VIP document

```
apiVersion: v1alpha1
kind: Layer2VIPConfig
name: 10.42.0.TBD
link: <CONTROL_PLANE_INTERFACE>
```

Platform worker labels and taints

```
machine:
  nodeLabels:
    aerealith.io/node-pool: platform
    topology.kubernetes.io/zone: pve-a
  nodeTaints:
    aerealith.io/node-pool: platform:NoSchedule
```

Application worker labels

```
machine:
  nodeLabels:
    aerealith.io/node-pool: application
    topology.kubernetes.io/zone: pve-a
```

Critical workload spread

```
topologySpreadConstraints:
- maxSkew: 1
  topologyKey: topology.kubernetes.io/zone
  whenUnsatisfiable: DoNotSchedule
  labelSelector:
    matchLabels:
      app.kubernetes.io/name: aerealith-api
- maxSkew: 1
  topologyKey: kubernetes.io/hostname
  whenUnsatisfiable: DoNotSchedule
  labelSelector:
    matchLabels:
      app.kubernetes.io/name: aerealith-api
```

B. Port and firewall matrix

Source	Destination	Port/protocol	Purpose	Default
Management VPN	Talos nodes	50000/TCP	Talos API	Allow named subnet only
Management VPN + nodes	K8s API VIP/CP	6443/TCP	Kubernetes API	Allow
Control planes	Control planes	2379-2380/TCP	etcd	CP /32 only
Control plane / monitoring	Nodes	10250/TCP	Kubelet	Approved sources
K8s nodes	K8s nodes	8472/UDP	Cilium VXLAN	Node set only
K8s nodes	K8s nodes	51871/UDP	Cilium WireGuard	Node set only
Workers	Workers	15008/TCP	Istio HBONE	Worker set only
cloudflared	Cloudflare	7844/TCP+UDP	Tunnel	Outbound only
Authorized app workers	CRDB nodes	26257/TCP	SQL/TLS	Worker IP set + pod policy
Monitoring	CRDB	8080/TCP	Metrics/admin UI	Monitoring/VPN only
Flux/nodes	Git/OCI	443/TCP	Git/image pull	Approved FQDNs
All systems	DNS/NTP	53 TCP/UDP, 123 UDP	Resolution/time	Approved services
PVE	PBS	8007/TCP	Backup	Backup clients only
Ordinary LAN	K8s/PVE/CRDB/ OpenBao	Any	No business need	DROP

C. Component maturity and selection

Project	Maturity / foundation	Use in Aerealith
Kubernetes	CNCF Graduated	Orchestration
Cilium	CNCF Graduated	CNI/network security
Istio	CNCF Graduated	Service mesh
Flux	CNCF Graduated	GitOps
Prometheus	CNCF Graduated	Metrics/alert rules
cert-manager	CNCF Graduated	Certificate automation
Harbor	CNCF Graduated	Self-hosted registry target
Falco	CNCF Graduated	Runtime detection
OpenTelemetry	CNCF Graduated	Telemetry standard/pipeline
SPIFFE/SPIRE	CNCF Graduated	Future workload-identity expansion if Istio identity is insufficient
Kubescape	CNCF Incubating	Posture/compliance scan
Velero	CNCF Sandbox (2026)	Optional K8s resource/PV backup layer
Kyverno	CNCF project; current policy engine baseline	Admission and image verification
Flagger	Flux ecosystem	Progressive delivery
OpenBao	Linux Foundation ecosystem, not selected for CNCF status	Runtime secrets/PKI
Talos Linux	Sidero Labs; Kubernetes-focused OS	Secure immutable node OS

CNCF maturity is a useful signal, not a substitute for threat modeling, version support, security advisories, or internal operational competence. CNCF defines Graduated as the most mature project stage and Incubation as a project demonstrating growing adoption and maturity. [\[R25\]](#)[\[R37\]](#)

D. Authoritative references

ID	Reference	URL
R1	Talos Security Checklist	https://docs.siderolabs.com/talos/v1.12/security/talos-security-checklist
R2	Talos default hardening and CIS compliance	https://docs.siderolabs.com/talos/v1.12/security/talos-default-hardening-and-cis-compliance
R3	Talos virtual shared IP	https://docs.siderolabs.com/talos/v1.13/networking/advanced/vip
R4	Talos Cilium deployment guide	https://docs.siderolabs.com/kubernetes-guides/cni/deploying-cilium
R5	Talos ingress firewall	https://docs.siderolabs.com/talos/v1.12/networking/ingress-firewall
R6	Talos disaster recovery	https://docs.siderolabs.com/talos/v1.10/build-and-extend-talos/cluster-operations-and-maintenance/disaster-recovery
R7	Cilium and Istio integration	https://docs.cilium.io/en/latest/network/servicemesh/istio/
R8	Cilium Kubernetes compatibility	https://docs.cilium.io/en/stable/network/kubernetes/compatibility/
R9	Istio ambient platform prerequisites	https://istio.io/latest/docs/ambient/install/platform-prerequisites/
R10	Istio 1.30.4 release line	https://istio.io/latest/news/releases/1.30.x/
R11	Istio supported releases	https://istio.io/latest/docs/releases/supported-releases/
R12	Flux SOPS guide	https://fluxcd.io/flux/guides/mozilla-sops/
R13	Flagger Istio progressive delivery	https://docs.flagger.app/main/tutorials/istio-progressive-delivery
R14	Kyverno 1.19 release	https://kyverno.io/blog/2026/08/20/announcing-kyverno-release-1.19/
R15	Falco Kubernetes deployment	https://falco.org/docs/setup/kubernetes/
R16	Kubescape CNCF project	https://www.cncf.io/projects/kubescape/
R17	Harbor CNCF project	https://www.cncf.io/projects/harbor/
R18	cert-manager graduation	https://www.cncf.io/projects/cert-manager/
R19	OpenTelemetry CNCF project	https://www.cncf.io/projects/opentelemetry/
R20	SPIFFE CNCF project	https://www.cncf.io/projects/spiffe/
R21	Kubernetes HA with kubeadm	https://kubernetes.io/docs/setup/production-environment/tools/kubeadm/high-availability/
R22	RKE2 CIS hardening guide	https://docs.rke2.io/security/hardening_guide
R23	CockroachDB certificate guidance	https://www.cockroachlabs.com/docs/stable/create-security-certificates-openssl
R24	CockroachDB backup monitoring	https://www.cockroachlabs.com/docs/stable/backup-and-restore-monitoring
R25	CNCF project maturity model	https://contribute.cncf.io/projects/lifecycle/

ID	Reference	URL
R26	HIPAA Security Rule	https://www.hhs.gov/hipaa/for-professionals/security/index.html
R27	GDPR Article 32	https://eur-lex.europa.eu/eli/reg/2016/679/art_32/oj/eng
R28	ISO/IEC 27001 overview	https://www.iso.org/isoiec-27001-information-security.html
R29	AICPA SOC 2 overview	https://www.aicpa-cima.com/topic/audit-assurance/audit-and-assurance-greater-than-soc-2
R30	Cloudflare Tunnel HA replicas	https://developers.cloudflare.com/cloudflare-one/networks/connectors/cloudflare-tunnel/configure-tunnels/tunnel-availability/deploy-replicas/
R31	Proxmox Backup Server features	https://www.proxmox.com/en/products/proxmox-backup-server/features
R32	Talos logging	https://docs.siderolabs.com/talos/v1.13/configure-your-talos-cluster/logging-and-telemetry/logging
R33	Talos SecureBoot	https://docs.siderolabs.com/talos/v1.13/platform-specific-installations/bare-metal-platforms/secureboot
R34	Talos disk encryption	https://docs.siderolabs.com/talos/v1.12/configure-your-talos-cluster/storage-and-disk-management/disk-encryption
R35	Cilium system requirements	https://docs.cilium.io/en/stable/operations/system_requirements/
R36	Tetragon on Kubernetes / Talos requirements	https://tetragon.io/docs/installation/kubernetes/
R37	CNCF Graduated and Incubating Projects	https://www.cncf.io/projects/